

Prototype Pollution

Prototype Pollution is a JavaScript-specific vulnerability class that closes out your web security series with a genuinely different mental model than everything before it — where SQLi/XXE/SSTI/command injection all exploit a *parser* interpreting data as instructions, prototype pollution exploits a **core language feature of JavaScript itself**: how objects inherit properties. It shows up in both client-side (DOM XSS) and server-side (Node.js, up to RCE) contexts, and connects directly back to your SSTI guide's object-chaining concept and your XSS/DOM knowledge.

1. The JavaScript Fundamental You Need First — Prototypal Inheritance

Every JavaScript object has an internal link to another object called its **prototype**, from which it inherits properties. When you access a property on an object, JavaScript checks the object itself first — and if the property isn't found there, it walks **up the prototype chain** looking for it.

```
let user = {};  
user.toString(); // works, even though `user` never defin  
ed toString()  
                // – it's inherited from Object.prototy  
pe
```

The critical fact this whole vulnerability class hinges on: almost every object in JavaScript ultimately inherits from a single shared object — `Object.prototype`. If you can add a property to `Object.prototype` itself, **every single object in the entire application** — past, present, and future — will suddenly appear to have that property, unless it explicitly defines its own version that overrides it.

```
Object.prototype.isAdmin = true;  
  
let randomObject = {};  
console.log(randomObject.isAdmin); // true – inherited fr  
om the  
                                     // now-polluted glob
```

```

    al prototype,
                                                    // even though this
    object never
                                                    // set isAdmin itsel
    f

```

This is the entire vulnerability in one demonstration: **polluting one shared, global object silently reshapes the apparent behavior of every unrelated object throughout the whole application.**

2. What "Prototype Pollution" Actually Means

Prototype pollution is a vulnerability that lets an attacker **inject arbitrary properties onto global object prototypes**, which the application (or a third-party library it uses) may then unknowingly **inherit and trust** as legitimate configuration or data.

How it typically arises: applications frequently perform **recursive object merges** — taking user-controllable input (a JSON body, URL parameters) and merging it into an existing internal object, often for configuration or state-update purposes. If the merge function doesn't filter out dangerous property names, and the attacker includes a key like `__proto__` in their input, the merge operation can be tricked into writing directly onto the shared prototype instead of onto the intended local object.

```

function merge(target, source) {
  for (let key in source) {
    if (typeof source[key] === 'object') {
      target[key] = merge(target[key] || {}, source[key]);
    } else {
      target[key] = source[key];
    }
  }
  return target;
}

merge({}, JSON.parse('{"__proto__": {"isAdmin": true}}'));
// The recursive merge walks into "__proto__", treats it as

```

```
just
// another object key to descend into, and ends up writing
isAdmin
// directly onto Object.prototype itself
```

This is genuinely the same root-cause pattern as every injection guide in this series: **user-controlled input reaching a context where it's given structural/interpretive power it was never meant to have** — here, the "structure" being manipulated is JavaScript's own object model rather than a SQL query or shell command.

3. The Two Required Ingredients — Source and Gadget

Every successful prototype pollution exploit needs both of these, and finding them is genuinely a two-step hunt:

```
Source → any user-controllable input that lets you inject properties
        into a prototype object (URL query strings, URL fragments,
        JSON request bodies, web messages via postMessage())

Gadget → a property that the application (or a library it uses)
        READS from an object and uses in an UNSAFE way — this is
        what actually turns "I polluted a property" into a real
        vulnerability (XSS, RCE, access control bypass)
```

Why both are strictly necessary: finding a source alone just proves you *can* pollute a property — it's not yet an exploit until you find some code elsewhere in the application that actually *reads* that specific property back and does something dangerous with it. This mirrors your SSTI guide's Detect → Identify → Exploit structure almost exactly: confirming the injection is step one, finding what to actually *do* with it is the separate, often harder, step two.

A property can't be a gadget if the object already defines it directly — an object's own explicitly-set property always takes precedence over anything inherited from the prototype chain, so only properties an object *doesn't* already have (and would otherwise fall back to inheriting) are viable gadget candidates.

PART 1: Client-Side Prototype Pollution

4. Detecting a Client-Side Source

Via the URL query string or fragment:

```
https://vulnerable-website.com/?__proto__[foo]=bar  
https://vulnerable-website.com/#__proto__[foo]=bar
```

Confirming it worked — via the browser console:

```
Object.prototype  
// If your injected `foo: "bar"` property appears in the re  
turned  
// object, you've confirmed a working prototype pollution s  
ource
```

Using DOM Invader (Burp's built-in browser extension) for automated discovery — genuinely the recommended practical approach, since manually hunting for sources and gadgets across a real, minified, bundled JavaScript codebase is extremely tedious. DOM Invader can automatically detect prototype pollution sources on a page and, where a known-dangerous gadget exists in a loaded library, can even auto-generate a working proof-of-concept payload for you.

5. Sanitization Bypass — Recursive Merge Weaknesses

A common (and commonly broken) developer mitigation attempt: stripping the literal string `__proto__` from user input before merging. The classic mistake — **failing to sanitize recursively**:

```
Input:  __pro__proto__to___.gadget=payload  
Sanitizer removes "__proto__" ONCE:  
Result:  __proto__.gadget=payload    ← still a valid pollution vector!
```

Because the sanitizer only stripped the dangerous substring **one time**, the leftover fragments on either side of the removed text happened to reassemble into the exact string it was trying to block — precisely the same "blocklist filtering is fragile" lesson repeated across your SQLi, XXE, and command injection guides, just with a JavaScript-specific bypass mechanic.

Alternative injection vector — via `constructor.prototype`:

```
myObject.constructor.prototype
// is EQUIVALENT to:
myObject.__proto__
```

If an application specifically blocks the literal string `__proto__` but doesn't also account for the `constructor.prototype` path to the exact same object, this provides a completely separate route to the identical outcome — worth always testing both forms.

6. Finding Client-Side Gadgets → DOM XSS

The most severe and common client-side outcome is chaining pollution into **DOM-based XSS**. Using the browser DevTools' Sources tab, look through loaded JavaScript (especially third-party libraries) for code that reads a property from a config-style object and passes it into a dangerous DOM XSS sink:

```
// Vulnerable library code:
if (config.transport_url) {
  let script = document.createElement('script');
  script.src = config.transport_url; // classic DOM XSS
  sink
  document.body.appendChild(script);
}
```

If `config` never explicitly sets its own `transport_url`, but the code still checks for and trusts that property, it becomes a viable gadget — the property will be **inherited** from a polluted `Object.prototype` if you set it there first.

Full working exploit example:

```
<script>
location="https://vulnerable-website.com/#__proto__[hitCall
back]=alert(document.cookie)"
</script>
```

This pollutes `Object.prototype.hitCallback` via the URL fragment. When a vulnerable third-party library later reads `hitCallback` off some unrelated object (finding nothing set locally, so falling back to the now-polluted prototype) and

passes it into something like `setTimeout()`, the attacker's JavaScript executes — full DOM XSS, delivered entirely through prototype pollution as the mechanism, with **zero traditional injection payload characters** (`<script>`, quotes, etc.) **anywhere in the visible request** — genuinely a stealthy delivery method compared to classic reflected/DOM XSS.

Web messaging as a source, tying to your Open Redirect/cross-origin

knowledge: if a page listens for `postMessage()` events and merges the received message data into application state without validation, this is another exploitable source — an attacker-controlled page (or an iframe) can send a crafted message containing pollution payload keys directly.

PART 2: Server-Side Prototype Pollution

7. Why Server-Side Is a Different, Often More Dangerous, Game

JavaScript's rise on the server (primarily **Node.js**) means the exact same language-level weakness exists there too — but exploiting it server-side is meaningfully trickier and riskier than client-side, for two specific reasons worth understanding:

a) You can't just refresh to reset state. In a browser, testing prototype pollution is low-risk — reload the page and you're back to a clean environment. On a server, a polluted `Object.prototype` can **persist across every subsequent request to that same server process**, potentially affecting other users, and there's no equivalent "refresh" — you can genuinely break or crash the application for everyone while testing, a real risk worth being explicit about before testing outside a dedicated lab.

b) Detection requires different techniques, since you don't have direct browser-console access to just type `Object.prototype` and look.

8. Detecting Server-Side Sources — The Reflected-Response Technique

The most straightforward detection approach: find an endpoint that accepts JSON and **reflects an object back** in its response — commonly something like a profile/account update feature:

```
POST /my-account/change-address
{ "address_line_1": "123 Test St", "postcode": "AB1 2CD" }
```

Response:

```
{ "address_line_1": "123 Test St", "postcode": "AB1 2CD",  
  "email": "...", ... }
```

Testing methodology — inject a pollution payload, then check if it echoes back on an UNRELATED subsequent request:

1. Submit: { "address_line_1": "test", "__proto__": {"polluted": "yes"} }
2. Send a COMPLETELY DIFFERENT, unrelated request that also returns a JSON object representing some object in the application
3. Check whether "polluted": "yes" appears in THAT response too – even though you never explicitly set it on that specific object
4. If it does, you've confirmed the pollution is global and persistent, affecting objects far beyond the one you originally submitted it to

9. Detecting Server-Side Sources — The Error-Based / Status-Code Technique

A subtler technique for when responses don't conveniently reflect a full object back to you:

```
// Simplified server error-handling logic:  
function handleError(err) {  
  let status = err.status || err.statusCode || 500;  
  // ... build error response using `status`  
}
```

This code reads a `status` / `statusCode` property **off the error object itself first**, but if neither is explicitly set, falls back to `500`. **Testing methodology:**

1. Trigger any error response, note the default status code returned
2. Attempt to pollute the prototype with your own status property:
 `__proto__.status = 599` (pick an obscure code, 400-599 range, unlikely to occur naturally)
3. Trigger the SAME error again
4. If the response now returns 599 instead of the original default, you've confirmed the pollution successfully influenced server-side logic, purely by observing status-code behavior – no reflected object needed at all

10. Server-Side Gadgets — From Config Override to Remote Code Execution

Server-side gadgets follow the exact same "find code that reads an unset property and trusts it" hunt as client-side — but the potential impact ceiling is dramatically higher, since server-side code routinely has access to the filesystem, child processes, and system commands.

Privilege escalation example: if application logic somewhere checks

`user.isAdmin` for access-control decisions, and no code path explicitly sets

`isAdmin: false` by default on regular user objects, polluting

`Object.prototype.isAdmin = true` can make **every user in the entire application** appear to have admin privileges — a direct, severe authorization bypass, conceptually similar in outcome to the IDOR/broken-access-control findings from your earlier guides, just reached via a completely different mechanism.

Remote Code Execution example — the most severe possible outcome: some Node.js libraries (particularly certain templating or child-process-spawning utilities) read configuration-style properties that, if attacker-controlled, can be abused to inject and execute arbitrary system commands on the server. This is genuinely the server-side prototype pollution equivalent of your Command Injection and SSTI guides' RCE chains — the delivery mechanism (prototype pollution) is different, but the destination (arbitrary OS command execution) is exactly the same devastating outcome, and the same Netcat reverse-shell catching technique from your Netcat guide applies identically once RCE is achieved this way.

11. Practical Testing Workflow

1. Identify potential sources:
 - URL query strings and fragments (client-side)
 - JSON request bodies accepted by any endpoint (server-side)
 - `postMessage()` listeners (client-side)
2. Confirm pollution:
 - Client-side: check `Object.prototype` directly in DevTools console, or let DOM Invader detect it automatically
 - Server-side: use the reflected-response technique (Section 8) or the error-based status-code technique (Section 9)
3. If sanitization appears present, test bypasses:
 - Non-recursive stripping (Section 5's `__proto__` trick)
 - The `constructor.prototype` alternative path

4. Hunt for gadgets:
 - Client-side: manually review loaded JS via DevTools Sources tab, or rely on DOM Invader's gadget-scanning features
 - Server-side: look for config-reading code with unsafe fallback defaults, especially around error handling, templating, or process-spawning logic
5. Chain source + gadget into a working exploit:
 - Client-side → typically DOM XSS
 - Server-side → typically access-control bypass or, in the worst case, RCE
6. CAUTION on server-side testing specifically: pollution can persist across requests and potentially crash the server entirely - always test in a dedicated lab environment first, and treat any real-target testing with real caution given you likely won't have Section 7's convenient "just restart the server" reset option available

12. Prevention

- Sanitize property keys BEFORE merging user-controlled input into any existing object - explicitly reject `__proto__`, `constructor`, and `prototype` as key names
- Sanitize RECURSIVELY, not just once - a single-pass strip is exactly what Section 5's bypass defeats
- Prefer an ALLOWLIST of permitted keys over a blocklist wherever feasible - the same allowlist-over-blocklist principle repeated throughout this entire series (SQLi, XXE, SSTI, command injection, open redirect)
- Use `Object.create(null)` to create objects with NO prototype at all where appropriate - such objects cannot inherit ANY polluted properties, structurally eliminating the risk for that specific object
- Use `Object.freeze(Object.prototype)` to prevent the prototype from being modified at all, as a global hardening measure
- Use modern, safe merge utilities/library versions - many popular JavaScript libraries (lodash, jQuery, and others) have had documented CVEs for exactly this vulnerability class in their merge/extend functions historically, and patched versions specifically guard against `__proto__`-based injection
- Patch identified prototype pollution SOURCES even without a currently known exploitable gadget - a gadget may exist in a library you haven't audited yet, or get introduced in a future dependency update, echoing the exact same "fix it even if you can't fully weaponize it yet" principle from your Business Logic Vulnerabilities guide

13. Practical Lab Example

PortSwigger's Web Security Academy has a dedicated Prototype Pollution learning path covering both client-side and server-side labs in depth, with DOM Invader specifically built to support hands-on practice for the client-side portion.

Suggested practice order:

1. Enable DOM Invader in Burp's embedded browser
2. Work through a client-side source-detection lab first - confirm pollution via the Object.prototype console check manually before letting DOM Invader automate it
3. Move to a client-side gadget-hunting lab, chaining into DOM XSS
4. Switch to server-side labs - practice the reflected-response detection technique (Section 8) on the classic "change address" lab pattern
5. Finish with the server-side RCE lab, which walks through finding a command-execution gadget end to end - genuinely one of the most striking "wait, THAT'S how severe this gets" moments in the whole Academy

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-1321 (Improperly Controlled Modification of Object Prototype Attributes, "Prototype Pollution")
OWASP Top 10:2025	A05:2025 Injection (the pollution mechanism itself is an injection into object structure); severe gadget outcomes also touch A08:2025 Software or Data Integrity Failures
CVSS	Client-side DOM XSS outcomes typically score Medium/High; server-side RCE outcomes reach Critical, AV:N/PR:N/UI:N/C:H/I:H/A:H being a realistic profile for the worst-case chain
Cyber Kill Chain	Exploitation — client-side outcomes align with Delivery-stage XSS payload execution; server-side RCE outcomes align directly with the Exploitation → Installation transition from your Command Injection guide
MITRE ATT&CK	T1190 (Exploit Public-Facing Application) as the entry vector; T1059 (Command and Scripting Interpreter) once server-side RCE is achieved via a gadget

Quick Reference Cheat Sheet

Core concept: `Object.prototype` is SHARED by nearly every JS object -
polluting it affects everything that inherits from it

Detection (client-side):

?`__proto__[foo]=bar` or `#__proto__[foo]=bar`

→ check via DevTools console: `Object.prototype`

Detection (server-side):

1. Reflected-response: pollute via JSON body, check if it echoes back on an UNRELATED endpoint's response
2. Error-based: pollute a "status" property, trigger an error, check if the status code changed

Sanitization bypasses:

`__proto__to__`.gadget=payload (non-recursive strip bypass)

`myObject.constructor.prototype` (alternative to `__proto__`)

Two required ingredients:

SOURCE (can you inject a property?) + GADGET (does anything UNSAFELY READ that property?) = working exploit

Client-side impact: DOM XSS

Server-side impact: access control bypass → up to full RCE

Prevention:

Allowlist keys | Recursive sanitization | `Object.create(null)` |

`Object.freeze(Object.prototype)` | patched library versions